Federated Learning

March 3, 2026

1 Centralized Training Workflow

Traditional machine learning systems follow a centralized approach where data from many sources is gathered in one place to train a model [1]. As shown in Figure 1, data is first collected from various devices and locations, then sent to a central server, which is usually hosted on a cloud platform. The model is trained on this combined dataset, and once ready, it is sent back to the end devices for use [1].

In intelligent transportation systems, for example, buses, roadside sensors, and other infrastructure devices constantly produce data. This data is uploaded to cloud servers where it is processed and used for training [2]. The goal of centralized learning is to find a model that reduces the overall error across all the collected training samples. source :[3] .

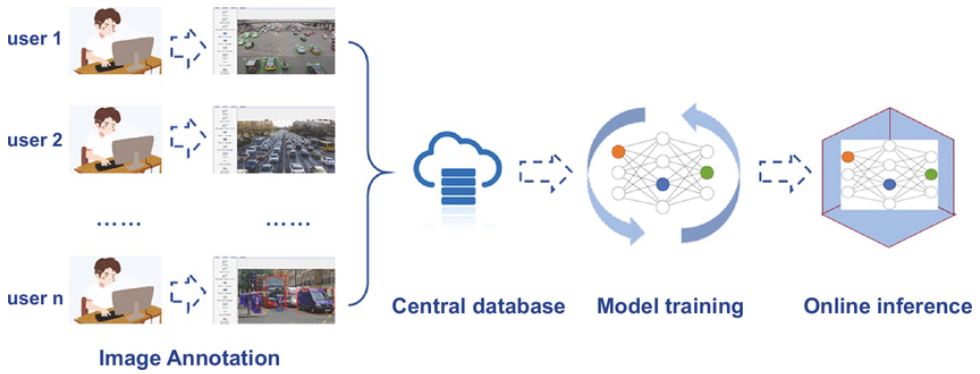

Figure 1: Centralized Training Workflow [4]

2 Limits of Centralized Machine Learning

Centralized machine learning faces several important limitations that make it poorly suited for modern transportation applications.

2.1 Privacy Vulnerabilities

Storing all data in one central location makes it an attractive target for attackers [5]. If that central storage is breached, personal or sensitive records can be exposed, which damages public trust in machine learning systems [5].

Beyond the risk of storing data in one place, the process of uploading data itself creates a vulnerability. When data travels from a bus or a vehicle to a remote cloud server, it passes through networks that can be intercepted. During this transit phase, attackers may capture sensitive information before it even reaches the central server. This means that data is at risk not only when it is stored but also while it is being transmitted [5].

In the context of bus fleets, for instance, on board sensors collect passenger counts and route information . Uploading all this to a single cloud server means that a single break could expose the travel patterns and habits of thousands of passengers at once.

2.2 Single Point of Failure

The central server is a single point of failure. If it goes down or is compromised, the entire system stops working, leading to service interruptions and loss of trust [2]. For a bus fleet management system, this would mean that all predictive maintenance, route optimization, and scheduling services could fail simultaneously [12].

2.3 Communication Costs

Sending large amounts of raw data to a central server comes with significant costs. These costs can be divided into two main categories [1].

First, there are **transport costs**, which refer to the bandwidth and network resources needed to move data from each device to the central server. When thousands of buses send sensor readings, video feeds, or diagnostic logs every day, the amount of data transferred becomes very large, and the cost of that data transfer grows accordingly [1].

Second, there are **storage costs**, which refer to the expense of keeping all that data on the central server. As the fleet grows and data accumulates over time, the storage infrastructure must be expanded continuously, adding to operational expenses [1].

2.4 Scalability Constraints

As data volume grows, centralized systems struggle to keep up. Adding more powerful hardware can help up to a point, but there are natural limits to how much a single system can handle [5]. When thousands of buses generate data at the same time, the processing load on one central entity becomes very heavy and difficult to manage [13].

2.5 Regulatory Compliance

In several countries and regions, regulations such as the General Data Protection Regulation (GDPR) in Europe impose strict rules on how personal data is collected, stored, and processed. These regulations make centralized data collection legally difficult because gathering all user data in one location increases the risk of non compliance [6]. Any organization handling private data must take great care to limit privacy risks, and centralized approaches make this harder to achieve [1].

2.6 The IID Assumption Problem

Centralized machine learning typically assumes that the training data is Independent and Identically Distributed (IID). This means that every data sample is assumed to come from the same underlying distribution, and each sample is independent from the others. In practice, this assumption rarely holds when data comes from different sources [3].

For example, buses operating in a city center will produce data that looks very different from buses running on suburban or rural routes. The driving patterns, passenger loads, road conditions, and even weather exposure vary greatly depending on the location and time of day. When all this data is mixed together and treated as if it comes from one uniform source, the resulting model may perform poorly on specific subgroups of the data [1]. This mismatch between the IID assumption and the reality of heterogeneous data is a fundamental limitation of centralized approaches.

3 Motivation for Decentralized Approach

The problems described above motivate the shift toward decentralized learning. In a decentralized approach, instead of gathering all data in one central server, the training process is spread across multiple devices or nodes. Each device works with its own local data, and only the results of the training (such as model updates) are shared, not the raw data itself.

There are two main types of decentralized approaches: **Distributed Learning** and **Federated Learning**. Although both involve training across multiple devices, they differ in their goals, architecture, and constraints.

3.1 Distributed Learning

Distributed learning focuses on making the training process faster by splitting the work across multiple machines, usually within a controlled environment such as a data center. The data is typically divided evenly among the machines, and they communicate through fast, reliable networks. All machines tend to have similar hardware capabilities. The main goal of distributed learning is to speed up training and handle larger datasets, not to protect privacy [8, 2].

As shown in Figure 2, the full dataset is divided into equal parts and sent to different worker machines. Each worker trains on its portion and sends the computed gradients back to a coordinator, which combines them to update the global model. The communication happens over high speed networks, and the data is typically assumed to be IID across workers [3].

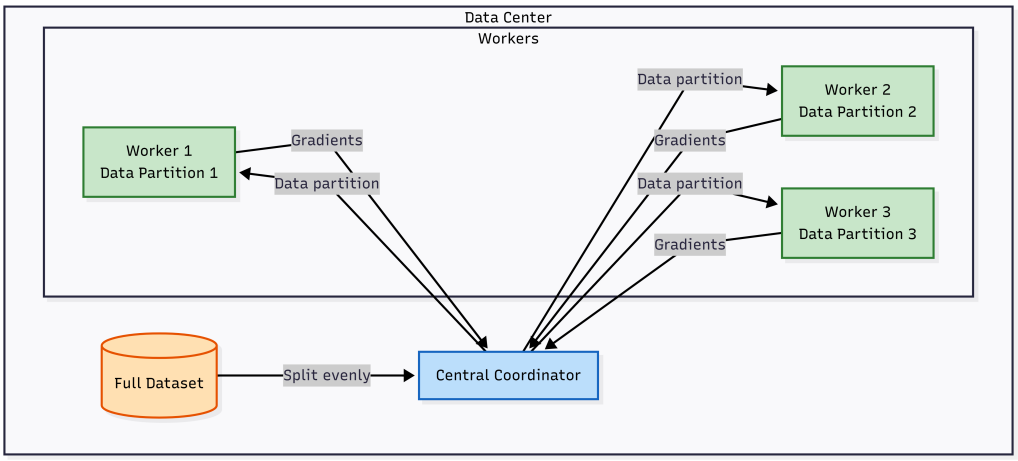

Figure 2: Distributed Learning Architecture (based on concepts described in [8]).

However, distributed learning has its own drawbacks. Since data must still be copied and sent to the worker machines, privacy is not protected. The approach also requires a controlled network environment with fast and reliable connections, which is not always possible in real world scenarios such as transportation systems. Furthermore, it does not handle situations where data is naturally non IID or where devices have very different hardware capabilities [8].

3.2 Federated Learning

Federated learning takes a different approach. Instead of moving data to the machines, the model is sent to where the data already lives. Each device trains the model locally on its own data and sends only the model updates back to a central server. The raw data never leaves the device [3].

As shown in Figure 7, in a federated learning setup applied to a bus fleet, each bus holds its own sensor data locally. The central server sends the current global model to the buses. Each bus trains the model on its local data and sends the updated model weights back to the server, which combines all the updates to improve the global model. The key advantage is that sensitive data such as passenger information, and GPS traces stays on the bus at all times [3, 1].

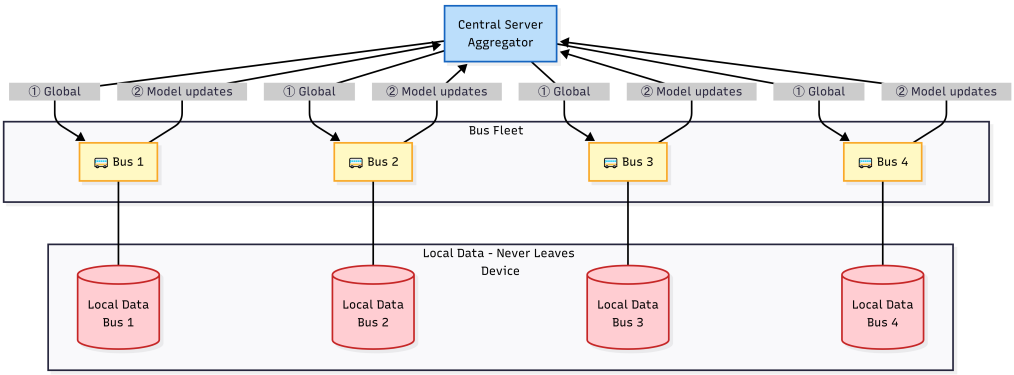

Figure 3: Federated Learning Architecture ([3]).

3.3 Advantages of federated learning

This approach brings several important benefits.

- **Privacy preservation.** Since data stays on the local device, sensitive information never travels across the network. For bus fleets, this means passenger data, route details, and operational logs remain protected without needing to be uploaded to a cloud server [7, 1].
- **Reduced communication overhead.** Instead of sending large volumes of raw sensor data, only compact model updates are transmitted. This significantly reduces the bandwidth needed, which is especially important for buses that may rely on mobile or intermittent network connections [8, 1].
- **Leveraging edge computing.** Modern on board computing units in buses and vehicles have enough processing power to train models locally without depending on cloud resources [3, 1].
- **Distributed knowledge aggregation.** Each bus operates under different conditions depending on its route, weather, traffic, and passenger load. Federated learning allows the global model to benefit from all this diversity without centralizing the data, leading to better generalization across the fleet [6, 14, 5].

4 Comparison of Learning Approaches

Table 1 summarizes the key differences between centralized, distributed, and federated learning.

Table 1: Comparison of Centralized, Distributed, and Federated Learning

Criteria	Centralized	Distributed	Federated
Data location	Central server	Split across workers	Stays locally
Privacy	Not protected	Not protected	Protected by design
Communication	High (raw data transfer)	High (fast internal network)	Low (only model updates)
Data distribution	Assumed IID	Typically IID	Non IID, unbalanced
Network requirements	Reliable internet	Reliable internet	Can work with slow or discontinuous connections
Scalability	Limited by server capacity	Good within data center	Scales to millions of devices
Main goal	Model accuracy	Training speed	Privacy preserving collaboration
Device heterogeneity	Not applicable	Homogeneous	Heterogeneous
Regulatory compliance	Difficult	Difficult	Easier

As shown in Table 1, federated learning provides a strong combination of privacy protection, scalability, and reduced communication compared to centralized and standard distributed approaches [6, 3].

As illustrated in Figure 4, the evolution of machine learning has followed a clear path. Centralized learning came first, but its scalability limits led to the development of distributed learning. However, distributed learning still does not address privacy concerns, which motivated the creation of federated learning [15].

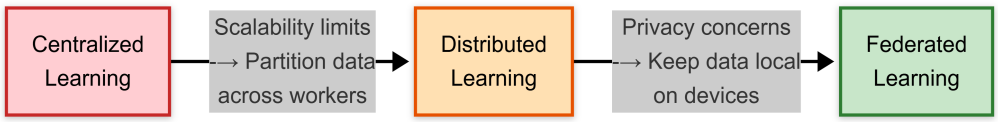

Figure 4: The evolution of machine learning paradigms ([15]).

5 Federated Learning: Concepts, Architecture, and Algorithms

5.1 Definition and Key Principles

Federated Learning is a machine learning approach where multiple clients work together to train a shared model, coordinated by a central server, while each client keeps its own data locally [3]. A global model is maintained by the server and improved over time through contributions from all participating devices [1].

The approach is built on several core principles [8]. First, raw data never leaves the client device, which is known as **data locality** . Second, only model updates are sent between devices and the server, making **communication model centric**. Third, all clients work together to optimize a shared objective through collaborative learning. Fourth, the architecture is designed from the start to limit data exposure, following a privacy by design philosophy. Fifth, the system can accommodate clients with different hardware capabilities and different data distributions, which is referred to as heterogeneity tolerance.

5.2 Types of Federated Learning

Federated learning can be categorized in two ways: by how data is distributed across clients, and by how the system is organized.

5.2.1 By Data Distribution

Federated learning can be categorized into three groups according to how the *samples* (data records) and the *features* (data attributes) are distributed across participants: **horizontal federated learning**, **vertical federated learning**, and **federated transfer learning** [24].
As shown in Figure 5:
(a) Horizontal FL (HFL): parties collect data in the same format (same features/columns), but each one has data from different users or events (different samples/rows). It is the most common setting when many devices or organizations train the same model on their own local data without sharing raw records [24].
(b) Vertical FL (VFL): parties are interested in the same users/IDs , but each party owns different information about them (different features/columns). The goal is to combine these complementary features while keeping them private, which usually requires matching the same user across parties (entity alignment) and using privacy-preserving training [24].
(c) Federated Transfer Learning (FTL): parties have little overlap in both users/IDs (samples) and in features, so they cannot directly apply HFL or VFL. Instead, transfer learning is used to share knowledge across different domains (e.g., different feature spaces or environments) while still training in a federated manner [24].

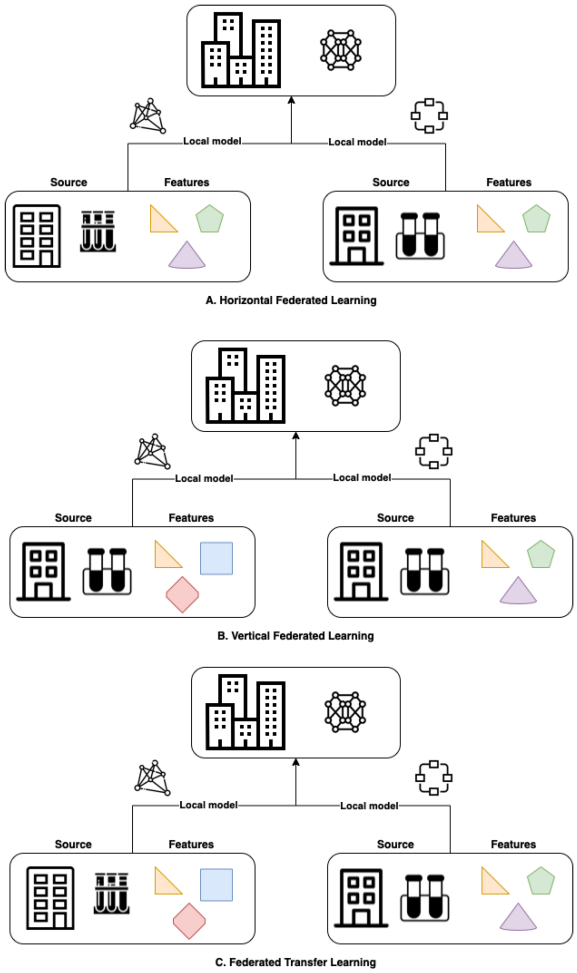

Figure 5: Data partition in (a) horizontal federated learning, (b) vertical federated learning, and (c) federated transfer learning. Source: [24].

5.2.2 By System Architecture

Federated learning (FL) can also be classified by **system architecture**, i.e., how coordination and aggregation are organized. In intelligent transportation systems (ITS), this choice affects scalability, latency,

and robustness under intermittent connectivity. As shown in Figure 6 , three common architectures are **centralized**, **hierarchical**, and **decentralized** FL [27, 28, 26].

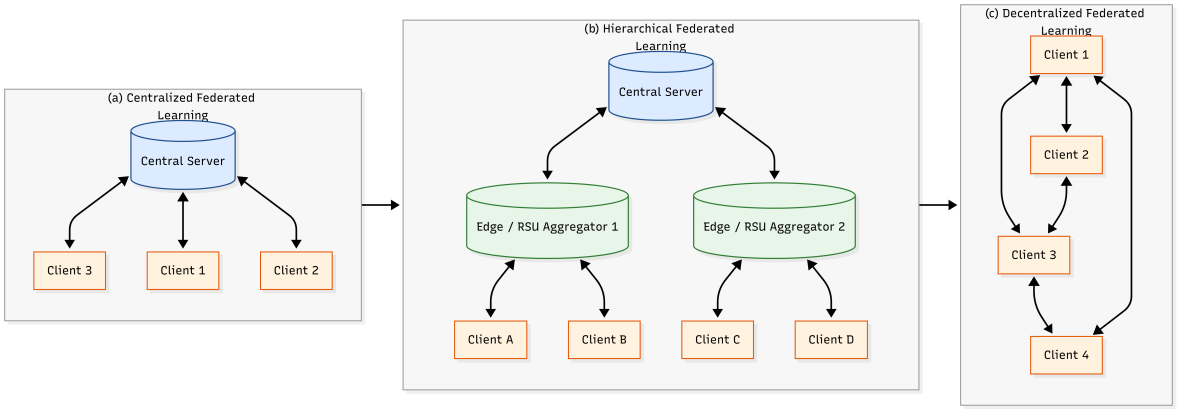

Figure 6: Comparison of federated learning system architectures: (a) centralized FL with a single coordinating server, (b) hierarchical FL

Centralized FL (Figure 6a) uses a single server to broadcast the global model and aggregate client updates, but it may become a bottleneck or a single point of failure [27, 28].
Hierarchical FL (Figure 6b) adds edge/RSU aggregators that perform intermediate aggregation before forwarding updates to the cloud, reducing backhaul load and latency in vehicular settings [28, 26].
Decentralized FL (Figure 6c) removes the central server and exchanges updates peer-to-peer over a communication graph, improving robustness but requiring more coordination and depending on connectivity [25, 29].

5.3 General Architecture of a Federated System

A federated learning system is made up of several components that work together [9].

The **central server**, also called the aggregator, is responsible for coordinating the entire learning process. It maintains the global model, collects updates from clients, aggregates them, and manages which clients participate in each training round.

The **clients**, or participants, are the devices that hold local private data. Each client trains the model on its own data and computes updates that are sent back to the server. Clients can be very different from each other in terms of computing power, network speed, and the amount of data they hold.

The **communication infrastructure** connects the server and the clients. It must be able to handle situations where connections are slow or interrupted, which is common in transportation scenarios. In some setups, edge nodes or roadside units act as intermediate aggregators to improve efficiency.

Finally, a **security and privacy layer** adds extra protection. This can include techniques such as secure aggregation, where the server can only see the combined result of all updates, or differential privacy, which adds noise to updates to prevent information leakage.

Figure 7 illustrates the complete architecture of a federated learning system with all its components.

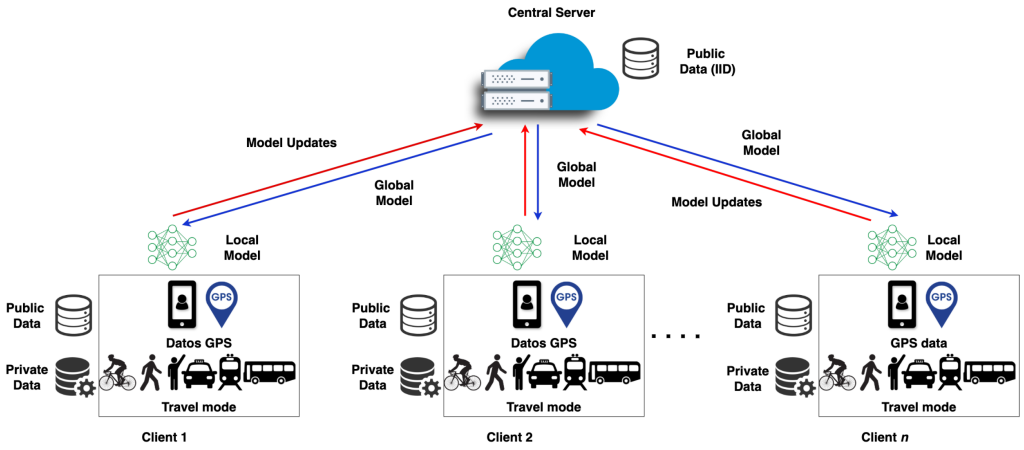

Figure 7: General architecture of a federated learning system. Source: [31].

5.4 Federated Learning Lifecycle

The federated learning process works in synchronous rounds, meaning that in each round, the server waits for all selected clients to finish their local training and send back their updates before moving on to the next round. The word "synchronous" here means that all participants follow the same schedule: no one moves ahead until everyone in the current round has completed their work [3].

In each round, the server first selects a group of available clients. It then sends the current version of the global model to those selected clients. Each client trains the model on its own local data for a number of steps. After local training, the clients send their updated models back to the server. The server then combines all the received updates, typically using a weighted average based on how much data each client has, to produce a new and improved global model. This cycle repeats until the model reaches a satisfactory level of performance.

In practical terms, a base model is created by the server and pushed to participating devices. Each device updates the model using its own data through an optimization process such as stochastic gradient descent, then returns the updated weights to the server. The server aggregates all the weight updates and uses them to refine the base model [1].

This synchronous approach is simple and effective, but it has a limitation: if one client is slow or disconnects, the entire round is delayed. To address this, some algorithms use an asynchronous approach instead, where the server does not wait for all clients but processes updates as they arrive. Asynchronous methods are better suited for real world scenarios like bus fleets, where vehicles may lose connectivity at any time [6].

Figure 8 illustrates the complete lifecycle of one federated learning round.

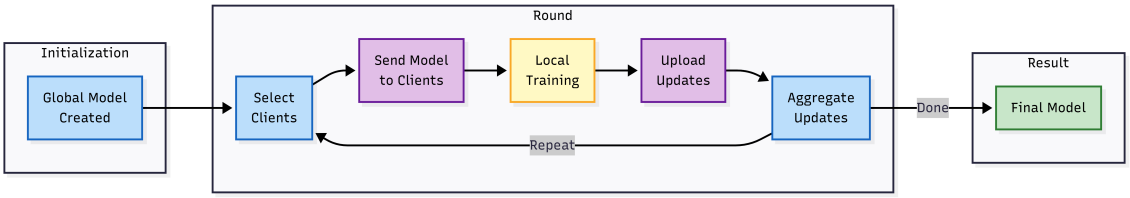

Figure 8: Federated Learning Lifecycle . source :[3]).

5.5 Core Algorithms

Several algorithms have been developed for federated learning, each addressing different challenges. This subsection presents the main ones.

5.5.1 Federated Averaging (FedAvg)

FedAvg is the first and most widely used federated learning algorithm, introduced by McMahan et al. [3]. The server initializes a global model and, in each communication round, selects a subset of clients,

sends them the current model, and receives locally trained updates. Each client runs local SGD for a few epochs on its private data, then returns the updated model; the server aggregates them using a weighted average proportional to each client's dataset size.

Compared to synchronized SGD, FedAvg can significantly reduce communication by allowing multiple local steps between aggregations and can perform well even under non-IID data [3]. However, strong client heterogeneity may slow convergence or cause instability, and theoretical guarantees are limited in general non-convex settings [20]. When clients take only one step using their full local data before averaging, FedAvg reduces to FedSGD [3, 1].

5.5.2 FedSGD

FedSGD is a special case of FedAvg where each client computes a single gradient step on its local data and sends it to the server. This is the simplest form of federated learning but requires more communication rounds since each round involves only one gradient computation per client [3, 1].

5.5.3 FedProx

FedProx was designed to handle heterogeneity better by adding a penalty term to the local training objective. This term prevents each client's model from drifting too far from the global model during local training, which improves stability when clients have very different data or computing capabilities [20].

5.5.4 DAAFL (Disparity Aware Asynchronous FL)

DAAFL addresses the problem of data disparity, where different clients have very different amounts of data. It dynamically adjusts the weight given to each client's contribution based on both the amount of data it has and how often it has been included in past rounds. This ensures that clients who have been underrepresented get a stronger voice in shaping the global model [6].

5.5.5 FedSA

FedSA is a semi asynchronous approach that begins with a synchronous phase, where all clients train and report back together, and then transitions to an asynchronous phase, where the server accepts updates as they come in without waiting for everyone. The number of local epochs each client performs is adjusted based on how outdated its model is compared to the current global model [21, 6].

5.5.6 SCAFFOLD

SCAFFOLD uses a technique called control variates to correct the drift that occurs when clients train on their own non IID data. This leads to better convergence, especially in situations where the data distributions across clients are very different, such as in transportation deployments where each bus encounters unique conditions [22].

5.5.7 FedNova

FedNova solves a problem that arises when different clients perform different numbers of local training steps. Without correction, this inconsistency can lead to a biased global model. FedNova normalizes the local updates to remove this bias, which is important in heterogeneous environments where devices have different processing speeds [23].

5.5.8 Convergence Considerations

For synchronous federated learning with well behaved objective functions, the speed of convergence depends on the number of participating clients and the number of local steps each one takes [25]. For asynchronous approaches, convergence is also affected by how outdated the client models are and how unevenly data is distributed across clients [6].

Non IID data generally slows down convergence. Because clients have different amounts of data and converge at different speeds, it is difficult to decide when to stop training. Traditional early stopping methods, which monitor a single validation metric, are not directly applicable in the federated setting [20, 6].

5.6 Privacy and Security Mechanisms

Even though raw data stays on the device, the model updates themselves can still reveal information about the underlying data. Research has shown that it is possible to reconstruct training data from shared gradients [16]. To address this, several defense mechanisms have been developed.

Differential Privacy (DP) works by adding controlled noise to the model updates before they are sent to the server. This makes it much harder for an attacker to reconstruct specific training examples from the updates. The trade off is that too much noise can slow down learning or reduce the final model accuracy [28, 17].

Secure Multi-Party Computation (SMPC) uses cryptographic techniques to allow the server to aggregate client updates without ever seeing the individual parameters in plain text. This means the server can compute the weighted average of all updates without knowing what any single client contributed [28, 18].

Homomorphic Encryption allows computations to be performed directly on encrypted data. Clients can encrypt their model updates before sending them, and the server can aggregate the encrypted values without decrypting them first. Only the final aggregated result is decrypted [19].

Blockchain Integration has also been proposed as a way to strengthen the integrity of the federated learning process. In this approach, each round of model updates is recorded as a transaction on a private blockchain, creating a permanent and tamper proof record of the entire training history. This prevents any participant from secretly modifying past updates [29].

However, adding these protections comes with trade offs. More security typically means slower training or lower model accuracy, because the added noise, encryption overhead, or verification steps affect the overall learning process [1].

6 Federated Learning for Predictive Maintenance

6.1 Application Context

Predictive maintenance in transportation fleets uses sensor data to anticipate equipment failures before they happen [7]. Federated learning is well suited for this task because it addresses several critical requirements at once: vehicle sensor data remains on board for privacy, each vehicle contributes knowledge from its own operating conditions, regulatory requirements are met since data stays local, and the system scales naturally across large fleets [6].

6.2 Challenges in Transportation FL

Transportation fleets face specific challenges when applying federated learning. Each vehicle produces data at different rates depending on how often and how far it travels. Buses on busy urban routes generate much more data than those on infrequent suburban lines. Additionally, vehicles may lose connectivity during operation, especially in tunnels, rural areas, or underground terminals, causing them to drop out of training rounds [6].

6.3 FL Based Predictive Maintenance Framework

In an industrial setting, the framework can be illustrated through autonomous guided vehicles (AGVs) operating in separate production facilities, each equipped with independent systems [7].

The data flow works as follows. Each vehicle collects sensor readings including energy consumption, motor activity, and distance traveled. Local models, typically based on architectures such as LSTM or GRU, are trained on the device using this data. Only the trained model weights are sent to the server, not the raw sensor data. The server then aggregates the received updates using a weighted averaging scheme based on validation performance [7].

6.4 Model Averaging Strategy

How local models are combined into a global model has a significant impact on performance. The server must carefully determine the influence of each local model using metrics such as mean squared error or validation loss. Giving appropriate weight to each contribution ensures that the global model benefits from all participants without being dominated by any single one [7].

References

[1] Ibrahim Abdul Majeed, Sagar Kaushik, Aniruddha Bardhan, Venkata Siva Kumar Tadi, Hwang-Ki Min, Karthikeyan Kumaraguru, and Rajasekhara Duvvuru Muni, "Comparative assessment of federated and centralized machine learning," in *arXiv preprint arXiv:2202.01529*, 2022.

[2] Georgios Drainakis, Konstantinos V. Katsaros, Panagiotis Pantazopoulos, Vasilis Sourlas, and Angelos Amditis, "Federated vs. centralized machine learning under privacy-elastic users: A comparative analysis," in *2020 IEEE 19th International Symposium on Network Computing and Applications (NCA)*, pp. 1-8, 2020.

[3] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*, pp. 1273-1282, PMLR, 2017.

[4] Yang Liu, Anbu Huang, Yun Luo, He Huang, Youzhi Liu, Yuanyuan Chen, Lican Feng, Tianjian Chen, Han Yu, and Qiang Yang, "Federated learning-powered visual object detection for safety monitoring," in *Special Topic Article, DOI: 10.1609/aaai.12002*, 2021.

[5] Glory Nosawaru Edege and Samuel Acheme, "A systematic review of centralized and decentralized machine learning models: Security concerns, defenses and future directions," in *NIPES Journal of Science and Technology Research*, vol. 6, no. 4, pp. 161-175, 2024.

[6] Leonie von Wahl, Niklas Heidenreich, Prasenjit Mitra, Michael Nolting, and Nicolas Tempelmeyer, "Data disparity and temporal unavailability aware asynchronous federated learning for predictive maintenance on transportation fleets," Volkswagen Group Technical Report, 2023.

[7] Bohdan Shubyn, Daniel Kostrzewa, Piotr Grzesik, Pawel Benecki, Taras Maksymyk, Vaidy Sunderam, Jia-Hao Syu, Jerry Chun-Wei Lin, and Dariusz Mrozek, "Federated learning for improved prediction of failures in autonomous guided vehicles," in *Journal of Computational Science*, 2023.

[8] Konstantinos Lazaros, Dimitrios E. Koumadorakis, Aristidis G. Vrahatis, and Sotiris Kotsiantis, "Federated learning: Navigating the landscape of collaborative intelligence," in *Electronics*, vol. 13, p. 4744, 2024.

[9] Mario S. Vela Romo, Carolina Tripp-Barba, Xavier Calderon Hinojosa, Pablo Barbecho, Luis Urquiza-Aguiar, and Nathaly Orozco, "Federated learning frameworks for intelligent transportation systems: A comparative adaptation analysis," in *MDPI*, 2025.

[10] Tariq Alqubaysi, Ammar F. A. Ahmed, Ahmed Almutairi, Faisal AlAnazi, Abdullah Asmari, and Ammar Armghan, "Federated learning-based predictive traffic management using a contained privacy-preserving scheme for autonomous vehicles," in *MDPI Sensors*, vol. 25, p. 1116, 2025.

[11] Hamad M. H. A. Alzari, Saif J. Alshamsi, Muhammad Adnan Khan, and Adnan Akhunzada, "Privacy-preserving interpretability: An explainable federated learning model for predictive maintenance in sustainable manufacturing and industry 4.0," in *MDPI*, 2025.

[12] Muhammad Asad, Ahmed Moustafa, and Takayuki Ito, "Federated learning versus classical machine learning: A convergence comparison," in *arXiv preprint arXiv:2107.10976*, 2021.

[13] Dana Alsagheer, Lei Xu, and Weidong Shi, "Decentralized machine learning governance: Overview, opportunities, and challenges," in *IEEE Access*, 2023.

[14] Georgios A. Kaissis, Marcus R. Makowski, Daniel Rückert, and Rickmer F. Braren, "Secure, privacy-preserving and federated machine learning in medical imaging," in *Nature Machine Intelligence*, vol. 2, no. 6, pp. 305-311, 2020.

[15] Darshan Naik and Nikita Naik, "The changing landscape of machine learning: A comparative analysis of centralized machine learning, distributed machine learning and federated machine learning," in *UK Workshop on Computational Intelligence*, pp. 18-28, Springer, 2023.

[16] Jonas Geiping, Hartmut Bauermeister, Hannah Dröge, and Michael Moeller, "Inverting gradients–how easy is it to break privacy in federated learning?" in *arXiv preprint arXiv:2003.14053*, 2020.

[17] Cynthia Dwork, Aaron Roth, et al., "The algorithmic foundations of differential privacy," in *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3-4, pp. 211-407, 2014.

[18] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1175-1191, 2017.

[19] Stephen Hardy, Wilko Henecka, Hamish Ivey-Law, Richard Nock, Giorgio Patrini, Guillaume Smith, and Brian Thorne, "Private federated learning on vertically partitioned data via entity resolution and additively homomorphic encryption," in *arXiv preprint arXiv:1711.10677*, 2017.

[20] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith, "Federated learning: Challenges, methods, and future directions," in *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50-60, 2020.

[21] Mingzhe Chen, Boming Mao, and Ting Ma, "FedSA: A staleness-aware asynchronous federated learning algorithm with non-IID data," in *Future Generation Computer Systems*, vol. 120, pp. 1-12, 2021.

[22] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *International Conference on Machine Learning*, pp. 5132-5143, PMLR, 2020.

[23] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H. Vincent Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 7611-7623, 2020.

[24] A. Bény, E. Kordík, J. Kratochvíl, and I. Zolotárová, "Federated Learning for Edge Computing: A Survey," *Applied Sciences*, vol. 12, no. 18, p. 9124, 2022.

[25] Sebastian U. Stich, "Local SGD converges fast and communicates little," in *arXiv preprint arXiv:1805.09767*, 2018.

[26] Mario S. Vela Romo, Carolina Tripp-Barba, Nathaly Orozco Garzón, Pablo Barbecho, Xavier Calderón Hinojosa, and Luis Urquiza-Aguiar, "Federated learning frameworks for intelligent transportation systems: A comparative adaptation analysis," *Smart Cities*, vol. 9, no. 12, 2026.

[27] Joahannes R. F. Souza, Stefano Z. L. N. Oliveira, and Helio Oliveira, "The impact of federated learning on urban computing," *Journal of Internet Services and Applications*, vol. 15, no. 1, 2024.

[28] Vishnu Pandi Chellapandi, Liangqi Yuan, Christopher G. Brinton, Stanislaw H. Żak, and Ziran Wang, "Federated learning for connected and automated vehicles: A survey of existing approaches and challenges," *IEEE Transactions on Intelligent Vehicles*, early access, 2023.

[29] Sergey Trofimov, Leonid Voskov, and Mikhail Komarov, "Decentralized public transport management system based on blockchain technology," *Applied Sciences*, vol. 15, no. 3, p. 1348, 2025.

[30] Hamad M. H. A. Alshkeili, Saif J. Almheiri, and Muhammad Adnan Khan, "Privacy-preserving interpretability: An explainable federated learning model for predictive maintenance in sustainable manufacturing and industry 4.0," *AI*, vol. 6, no. 6, p. 117, 2025.

[31] Yanbin Zhao, "Robust federated learning approach for travel mode identification from non-IID GPS trajectories," 2022.